



BITS Pilani
DIGITAL
Excellence made yours

Data Engineering for AI Systems Module 2: Week 5

Balu Gopalakrishna Pillai

Recap of Week 4 and Quiz

Sections

1. Introduction — labelling, storage, versioning and knowledge representations
 2. Labelling
 3. Activity 1: Labelling the given dataset
 4. Storage and access
 5. Dataset versioning
 6. Knowledge graph and embeddings
 7. Activity 2: Building a knowledge graph
 8. Closing
-

Section 1: Introduction — Labelling, Storage, Versioning and Knowledge Representations

Turning Raw Data Into Institutional Memory

Once data is stored, labelled and versioned, it begins to shape:

- How decisions are made
- What patterns are considered “normal”
- What risks are emphasised or ignored
- How future systems will learn

This is how we encode your organisation's understanding of the world!

What is Institutional Memory?

It is the accumulation of:

- What has happened before
- How it was interpreted
- Which decisions were made and why
- What patterns are considered meaningful

Modern AI systems externalise that memory into:

- Labelled datasets
- Feature stores
- Vector representation
- Knowledge graphs
- Versioned corpora

These systems allow organisations to learn consistently over time

Learning Objective

By the end of this week, you should be able to:

- Design labelling workflows that are consistent, reviewable and scalable
- Reason about labelling noise and understand how disagreement affects models
- Version datasets meaningfully, not just overwrite them
- Explain vector databases versus traditional storage and when to use each
- Understand where knowledge graphs fit in representing structured organisational knowledge



Section 2: Labelling

Labelling — A Socio Technical System

Tag the image, classify the text and move on?

When you label, you bring:

- Bias
- Context
- Fatigue
- Interpretation
- Cultural assumptions
- Varying levels of expertise

This means labels are not data, they are decisions made by people, under constraints, at a particular moment in time

What Could Go Wrong in Labelling?

Inter-annotator disagreements reveal:

- Poorly defined categories
- Overlapping concepts
- Missing context
- Real-world uncertainty that the taxonomy is trying to flatten

Hidden risk of labelling drift:

- Guidelines evolve
 - Teams change
 - Edge cases accumulate
 - Annotators develop shortcuts
 - Business priorities shift
-

Label Encodes Values and not Truths

When you label data, you are making choices about:

- What matters
- What is acceptable
- What counts as success
- What constitutes risk
- What distinctions are worth modelling and which can be ignored

Labelling should be treated as knowledge design, not clerical work.

Section 3: Activity 1

Label The Given Dataset

Scenario: Company wants to build a model that automatically route support tickets.

Task: Label each customer support message into the category you think it belongs to:

Categories are:

1. Billing issue
2. Technical issue
3. Account issue
4. General inquiry
5. Urgent escalation

Dataset



ID	Customer Message
1	"I was charged twice this month."
2	"My login is not working."
3	"The app crashes whenever I upload a file."
4	"Can you tell me your pricing plans?"
5	"My account was suddenly locked."
6	"I need help urgently, my payment failed during checkout."
7	"Where can I download the invoice?"
8	"The system is extremely slow today."
9	"I want to change my subscription plan."
10	"Your product is amazing!"

Labelling Guidelines

- Billing Issue → charges, invoices, payments
 - Technical Issue → software malfunction
 - Account Issue → login, access, account status
 - General Inquiry → information requests
 - Urgent Escalation → time-sensitive or critical issues
-
- What would happen if we trained a model on inconsistent labels?
 - What happens if the guidelines changes over time?
 - Who should define labels?

Section 4: Storage and Access

Raw Zone vs Curated Zone

Raw Zone:

- Immutable input data
- Never modify — only append; no cleaning, normalisation or transformation. This is the forensic layer.

Curated Zone:

- Cleaned, structured, validated, conformed to two scheme and enriched with derived features.
- Layer most models consume

Feature Store vs Embeddings

Feature store — a consistency mechanism that provides:

Shared featured definition, centralised transformation logic, Reusable computation pipelines, online/offline access to the same features.

Embeddings are derived data

- Compressed representations — mathematical summaries created for a specific model and purpose.
- They make similarity, retrieval or prediction computationally feasible.

Section 5: Dataset Versioning

Not Same as Code Versioning

Goal: Make historical experiments repayable

Dataset versioning must answer three questions:

- What changed? — new data, labelling, filtering or feature logic
- Why did it change? — bug fix, concept drift, policy update, expanded code
- How does it affect models? — invalidate past evaluations, require retraining, change metrics

Dataset versioning should track:

- Raw input sources and their snapshot in time
- Label sets and annotation guidelines used
- Preprocessing logic and parameters
- Feature definitions and transformations
- Validation rules applied
- Inclusion and exclusion criteria

Why Dataset Versioning Matters?

Especially long lived systems, to preserve context, not just content

Most AI systems are not trained once but evolve over the years.

- Teams change
- Definitions shift
- Data grows
- Objectives expand

Without structured versioning, we can't answer: "Why does the system behave differently today than it did last year?"

Section 6: Knowledge Graph and Embeddings

Knowledge Representation — Embeddings

- Captures implicit knowledge — makes them probabilistic, flexible, excellent for search and retrieval, robust to messy and unstructured data
- Do not store the facts directly, but store patterns of similarity learned from data
- Concepts that appear in similar contexts are placed near each other
- Meaning is statistical, not declarative
- Relationships are inferred, not defined
- They encode proximity

Knowledge Representation — Knowledge Graph

- Captures explicit knowledge
- Defines entities, relationships, constraints and ontologies, structured meaning that is human interpretable.
- Relationships are not inferred, they are declared.
- Supports logical reasoning, traceable relationships, domain constraints, controlled vocabularies and explainability

Where They Differ?

Finding vs Reasoning:

- An embedding model can retrieve documents similar to a query about a system failure.
- But a knowledge graph can explicitly show:
 - Which components depend on that system
 - Which teams own those components
 - What past incidents are related
 - What policies govern resolution

Where They Differ?

Drift vs Evolving:

- Embeddings are tightly coupled to the model that generated them.
- When you retrain the model, the representation changes. You can validate relationships against schemas
- Knowledge graphs evolve differently — they are extended, curated and versioned
- Knowledge graph behaves like long— lived infra for meaning

Usage:

- Embeddings for discovery, search and contextual mapping
- Knowledge graph for structure, governance and reasoning.

Section 7: Activity 2

Build a Knowledge Graph

Task: Build a Knowledge Graph for the hospital system

Objective: A hospital wants to build an AI system that understands relationships between patients, doctors, treatments, and diagnoses. Instead of tables, this will be represented by a knowledge graph

Concept: In a KG, we represent information as: Node - Relationship – Node

Eg:

Sunil (patient) -> consults (relationship) -> Dr Mehta (doctor)

Ravi (patient) -> is_diagnosed_with (relationship) ->diabetics (diagnosis)

Task:

- Identify entities (nodes)
- Identify relationships (edges)
- Draw a knowledge graph

Hospital Dataset

Patient	Doctor	Diagnosis	Treatment	Hospital
Ravi	Dr Mehta	Diabetics	Insulin	Manipal Hospital
Ananya	Dr Iyer	Hypertension	Food Control + Exercise	Metro Hospital
Ravi	Dr Khan	High Cholesterol	Statins	Vydehi Hospital
Sunil	Dr Latha	Asthma	Inhaler	Agarwal Hospital
John	Dr Mehta	Diabetics	Medicine	City Hospital

Section 8: Closing

Closing

- **Datasets outlive everything — they become institutional memory**
Models, teams and documentation change, but datasets persist and silently shape future decisions.
 - **Casual data practices create long—term, compounding risk**
Poor labelling, weak lineage, missing versioning and loose representations accumulate into bias, irreproducibility and loss of meaning.
 - **The biggest AI failures are slow and invisible**
Context erosion happens gradually until no one can explain system behaviour or trust its outputs.
 - **Data must be engineered — not treated as operational exhaust**
It requires intentional design, governance, lifecycle management and ownership.
 - **How your data remembers determines how your AI decides**
Reliable AI depends on reliable, traceable, well—governed datasets — this is where organisational memory begins.
-

Thank You
